HOUSE PRICE PREDICTION

Machine Learning Project Report

Project Type	Full-Stack Machine Learning Web Application
Machine Learning Model	Linear Regression
Backend	Flask REST API
Frontend	Streamlit
Dataset	House Price Dataset (area, rooms, price)
Author	Monu Khan

1. Abstract

The House Price Prediction project is an end-to-end machine learning application designed to estimate house prices from basic property information. The application uses a Linear Regression model trained on a house-price dataset containing area, number of rooms, and price. A Flask REST API provides the prediction service, while a Streamlit interface provides a simple user-facing frontend. The project demonstrates the complete flow from dataset preparation and model training to API integration and interactive prediction.

2. Introduction

House price prediction is a common supervised machine learning problem. Property prices can depend on factors such as area and number of rooms. In this project, these features are used to build a simple regression model that learns the relationship between property characteristics and price. The trained model can then be used to make predictions for new property inputs.

3. Objectives

- Build a working machine learning model for house-price prediction.
- Prepare and use a structured house-price dataset.
- Train a Linear Regression model using relevant input features.
- Save the trained model for later prediction.
- Create a Flask REST API for model inference.
- Create a Streamlit frontend for easy user interaction.
- Demonstrate an end-to-end ML application suitable for an academic/internship project.

4. Technologies Used

Technology	Purpose
Python	Core programming language
Pandas / NumPy	Data handling and numerical operations
Scikit-learn	Machine learning model and training

Technology	Purpose
Joblib	Saving/loading the trained model
Flask	REST API backend
Streamlit	Interactive web frontend

5. Dataset Description

The project uses a house-price dataset stored as **data/house_price.csv**. The project screen shown in the supplied reference indicates 47 samples with the main fields **area**, **rooms**, and **price**.

Column	Role	Description
area	Input feature	House area in square feet
rooms	Input feature	Number of rooms
price	Target	House price to be predicted

6. Methodology

- **Data Preparation:** Load the CSV dataset and select the input and target columns.
- **Model Training:** Train a Linear Regression model using area and rooms as input features and price as the target.
- **Model Persistence:** Save the trained model as a .pkl file so it can be reused without retraining.
- **Backend Integration:** Load the model in Flask and expose an endpoint for prediction requests.
- **Frontend Integration:** Use Streamlit to collect area and room values from the user and display the prediction.
- **Validation:** Verify that the model, backend, frontend, and project files are present and runnable.

7. System Architecture

User → Streamlit Frontend → Flask REST API → Trained Linear Regression Model → Predicted House Price

The frontend collects the property information. The backend receives the input, loads the saved model, performs inference, and returns the predicted value to the frontend for display.

8. Project Structure

```
house_price_project/
■■■ backend/
■ ■■■ app.py
■■■ data/
■ ■■■ house_price.csv
■■■ frontend/
■ ■■■ ui.py
■■■ model/
■ ■■■ model.pkl
■■■ report_images/
■■■ train_model.py
■■■ requirements.txt
■■■ README.md
```

9. Implementation and Execution

The model training script creates the trained Linear Regression model. The resulting model is stored in the model directory. The Flask application loads this model and serves predictions through an API. The Streamlit application provides an accessible interface for entering property details.

Typical commands

- `pip install -r requirements.txt`
- `python train_model.py`
- `python backend/app.py`
- `streamlit run frontend/ui.py`

10. Example Prediction Flow

- User enters the house area, for example 1200 sq ft.
- User enters the number of rooms, for example 3.
- Streamlit sends the input to the prediction backend.
- Flask passes the input to the trained Linear Regression model.
- The model calculates the estimated house price.
- The predicted price is returned and displayed to the user.

11. Project Validation

The supplied project screenshot shows that the model training completed successfully and that the project files were checked. The visible project report summary describes the application as a Flask backend with a Streamlit frontend built around the house_price.csv dataset.

12. Project Screenshot

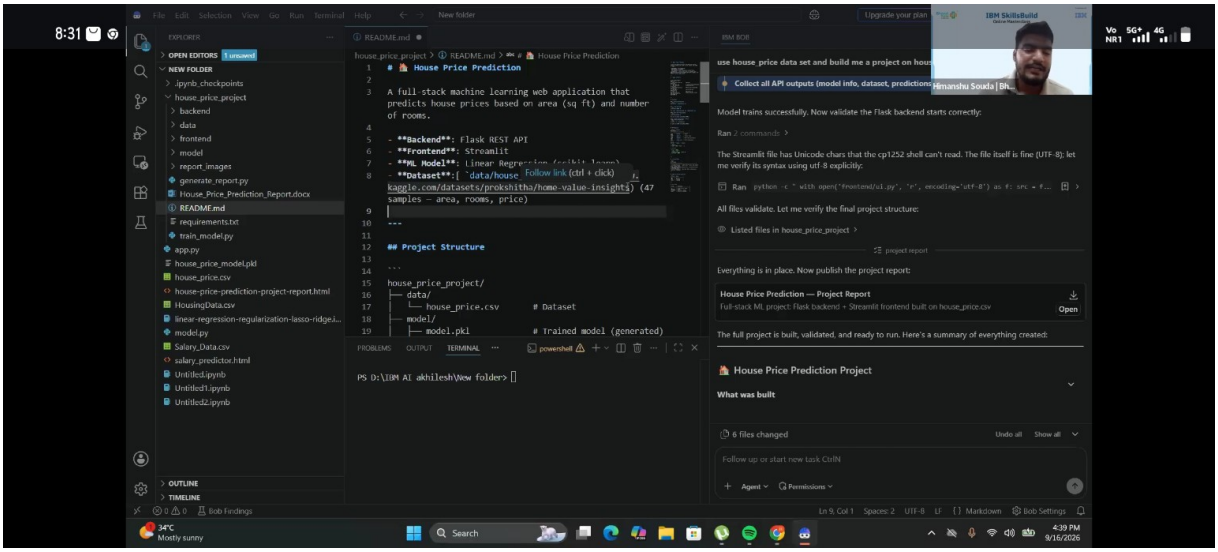


Figure 1: Reference screenshot showing the House Price Prediction project structure, README, model files, and project report interface.

13. Advantages

- Simple and easy-to-understand machine learning workflow.

- Interactive frontend makes prediction convenient for users.
- Flask API separates the prediction service from the user interface.
- Saved model can be reused without training every time.
- Project can be extended with additional property features and improved models.

14. Limitations

- The model uses a small and limited set of features.
- Linear Regression may not capture complex relationships in real-world property prices.
- Prediction quality depends strongly on the quality and representativeness of the dataset.
- A larger real-world dataset and additional validation would be needed for production use.

15. Future Scope

- Add location, property age, bathrooms, floor, and other relevant features.
- Compare Linear Regression with Random Forest, Gradient Boosting, or other regression models.
- Add model evaluation metrics such as MAE, MSE, RMSE, and R^2.
- Deploy the Flask API and Streamlit application online.
- Add charts and visual analytics for better understanding of predictions.

16. Conclusion

The House Price Prediction project successfully demonstrates how machine learning can be integrated into a complete web application. By combining a Linear Regression model with a Flask REST API and Streamlit frontend, the project provides a clear example of an end-to-end machine learning solution. It is suitable for learning, academic demonstration, and internship project submission, while also providing a foundation for future improvements.

End of Project Report